

Python sys.monitoring 简介

面向 Tracing / Debugging / Profiling 的新一代运行时观测接口

背景：为什么需要 sys.monitoring ?

- sys.settrace 存在的问题：
 - 事件粒度粗（行级）
 - 全局单一回调，难以组合多个工具
 - 性能开销大
- Python 3.12 引入 sys.monitoring
- 目标：
 - 更细粒度事件
 - 多工具并存（debugger / profiler / tracer）
 - 更低运行时开销

- 事件驱动模型 (Event-based)
- 为不同工具分配 **tool_id**
- 每个工具订阅自己关心的事件

典型事件类型：

- PY_START / PY_RETURN
- LINE
- CALL / C_CALL
- INSTRUCTION

Tracer 示例：函数调用追踪

```
import sys
from sys import monitoring

TOOL_ID = 1

def on_call(code, offset):
    print(f"CALL {code.co_name}")

monitoring.use_tool_id(TOOL_ID, "simple-tracer")
monitoring.register_callback(TOOL_ID,
                             monitoring.events.PY_CALL, on_call)
monitoring.set_events(TOOL_ID, monitoring.events.PY_CA

def foo():
    return 42

foo()
```

Debugger:

- 监听 LINE / INSTRUCTION 事件
- 支持断点、单步执行

Profiler:

- 监听 CALL / RETURN
- 构建调用栈、统计时间

Tracer 示例代码:

```
# 调试器和性能分析器可以同时启用  
# 而不会覆盖彼此的回调逻辑
```

sys.monitoring vs sys.settrace

维度	sys.settrace	sys.monitoring
事件粒度	行级	行 / 指令 / 调用
多工具支持	否	是 (tool_id)
性能	较高开销	更低
设计年代	Python 早期	Python 3.12+
适用场景	简单调试	可观测性 / 运行时分析

sys.monitoring 支持更高性能的 tracing

可同时用于调试和性能分析

总结

- `sys.monitoring` 是 Python 运行时可观测性的基础设施
- 更适合：
 - 高性能 tracing
 - 组合式调试与分析工具
 - 系统级行为建模
- 可作为：
 - 动态分析工具
 - 运行时剖析框架
 - 可观测性驱动优化的底层支撑

使用 `fragile + verbatim`, Python f-string 安全显示